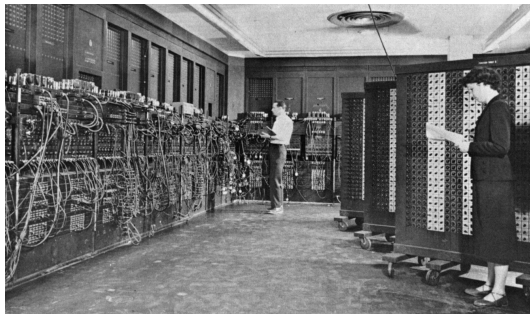
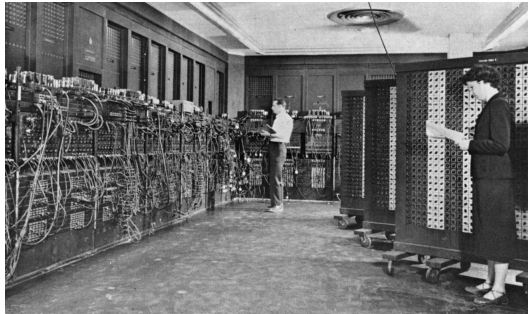


Origines



Ordinateur ENIAC à Aberdeen Proving Ground, Maryland, É.-U. A. Glen Beck et Betty Snyder programment l'ordinateur dans le bâtiment 328 du laboratoire de recherche balistique (U.S. Army, domaine public, par Wikimedia Commons).

Origines



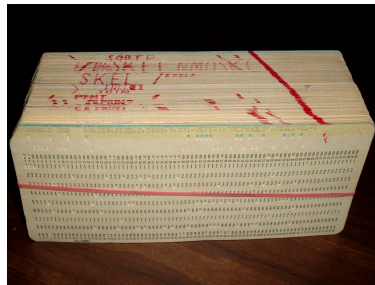
Ordinateur ENIAC à Aberdeen Proving Ground, Maryland, É.-U. A. Glen Beck et Betty Snyder programment l'ordinateur dans le bâtiment 328 du laboratoire de recherche balistique (U.S. Army, domaine public, par Wikimedia Commons).

- programmation par câblage manuel des unités
- lecture des résultats sur des indicateurs lumineux
- exécution d'un seul programme à la fois
- aucun système d'exploitation

Origines



Machine IBM 704 à Hampton, Virginie, É.-U. A. Un homme et une femme travaillent au centre de recherche Langley avec cette machine utilisée en recherche aéronautique (National Advisory Committee for Aeronautics, domaine public, par Wikimedia Commons).



Ensemble de cartes perforées représentant un programme écrit en langage PL/1 pour IBM 360 (Arnold Reinhold, Creative Commons BY-SA 3.0, par Wikimedia Commons).

- automatisation du passage des programmes
- traitement par lots par un programme moniteur résident en mémoire
- exécution d'un seul programme à la fois
- premiers embryons des systèmes d'exploitation

Origines

Multi-programmation

- séparation de la mémoire en partitions contenant des programmes différents
 - introduction d'une unité de gestion de la mémoire
- rentabilisation du temps mort lié aux entrées et aux sorties d'un programme
 - introduction d'un mécanisme d'interruptions

Spooling

- pré-chargement de programmes depuis les cartes perforées vers le disque
- démarrage du programme suivant dès la fin d'exécution du programme précédent
- problèmes de partage des ressources

Origines

Temps partagé

- accueil simultané de plusieurs utilisateurs
- allouer le processeur pour une petite quantité de temps à tour de rôles
 - quantum, par exemple 20 ms

Télétype

- saisie de commandes sur le clavier
- impression directe du résultat
- introduction d'un langage de commandes



Télétype ASR33 (AlisonW, Creative Commons BY-SA 3.0, par Wikimedia Commons).

Origines

Mise en place du temps partagé

- 1 reprise du contrôle à la fin de quantum
 - horloge avec interruption
- 2 gestion des utilisateurs
 - attribution d'identité
 - validation et autorisations
- 3 garanties sur la sécurité
 - modes d'exécution privilégié et non-privilégié

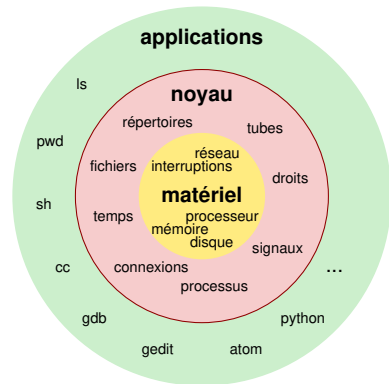
Appels système

- basculement en mode privilégié
- vérification des paramètres et des droits utilisateurs
- accès au disque pour lire un fichier, ...

Définition

Rôle

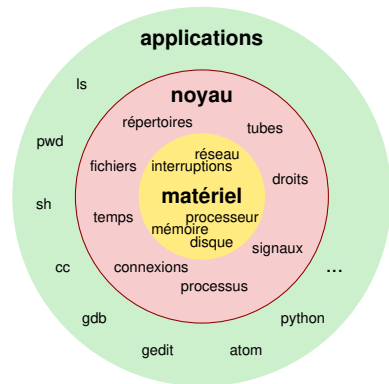
- 1 rentabiliser l'usage de l'ordinateur
- 2 partager équitablement les ressources matérielles
- 3 offrir une interface avec le matériel
- 4 permettre à des utilisateurs d'éditer, développer, lancer des programmes
- 5 offrir les garanties de sécurité nécessaires



Noyau

Objectifs

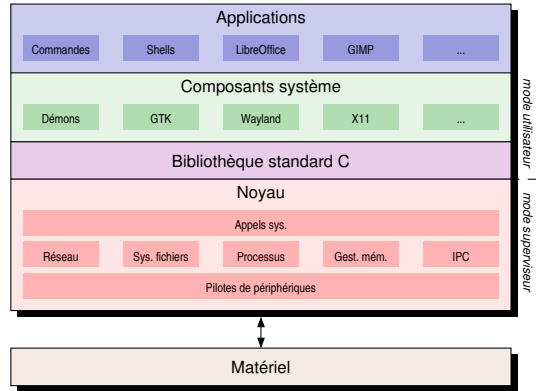
- 1 partager équitablement les ressources
 - mémoire vive
 - temps processeur
 - carte réseau
 - espace disque
 - ...
- 2 garantir la sécurité des données
 - protéger les données des processus
 - faire respecter les droits d'accès aux fichiers
 - ...



Noyau

Principes fondamentaux

- installé en mémoire dès le démarrage de l'ordinateur
- exécution en mode privilégié
 - accès à toutes les ressources matérielles
- mise à disposition des abstractions
 - contrôle d'accès aux ressources matérielles
- exécution des applications en mode non-privilégié
 - contexte de processus
 - appels au noyau via les primitives système



Vue schématique du noyau Unix/Linux.

Principales abstractions

- **processus** – co-existence de plusieurs applications en mémoire et partage du temps processeur
- **fichier et périphérique** – éviter des accès directs au matériel
- **utilisateur** – accueil de plusieurs utilisateurs sur un même ordinateur
- **tube** – communication entre processus sur le même système
- **socket** – communication entre processus sur des machines distinctes
- **signal** – gestion d'événements asynchrones

Familles

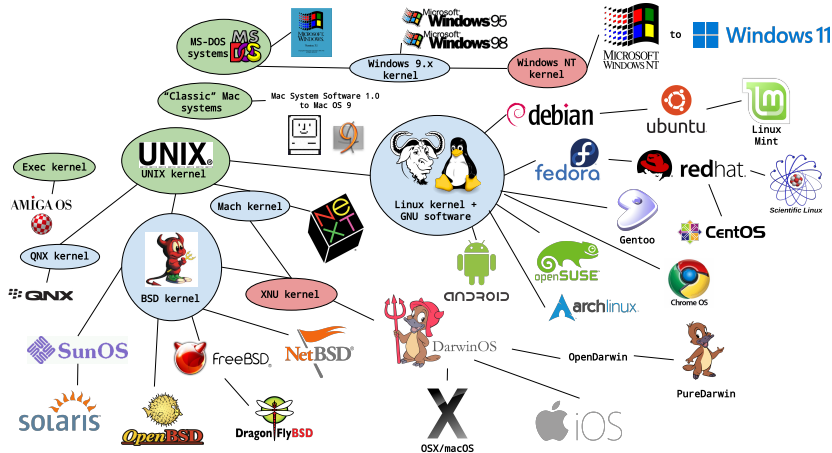


Schéma par Ethan Gates (Creative Commons BY-SA 3.0).



- basé sur la philosophie *kiss* « keep it simple, stupid »
- écrit en langage C et en assembleur
- séparation nette entre le noyau et les applications
- tout est fichier
- pas de structure interne des fichiers
- interface utilisateur simple
- portable
- **omniprésent dans le domaine du calcul haute-performance** à travers **GNU/Linux**

Primitives système

- appels au noyau depuis les applications
- accessibles à travers une interface de programmation en langage C

Exemples

- ouverture d'un fichier

```
int open(const char * path, int flag, mode_t mode);
```

- récupération de l'identifiant d'un utilisateur

```
uid_t getuid(void);
```

Portable Operating System Interface

Norme de programmation système

- primitives système et fonctions de bibliothèque
- commandes (sh, ls, tr, ...)
- extensions (temps réel, fils d'exécution, sémaphores, échange de messages ...)

Norme IEEE Std 1003.1

- version actuelle date de 2024
- <https://pubs.opengroup.org/onlinepubs/9799919799/>